

Interactive Schwarzschild Black Hole Simulator

Technical Architecture Blueprint

Author: Natalia
Project Type: Interactive Scientific Simulation
Target Platform: Web Browser

1. Architectural Philosophy

The simulator follows three strict engineering principles:

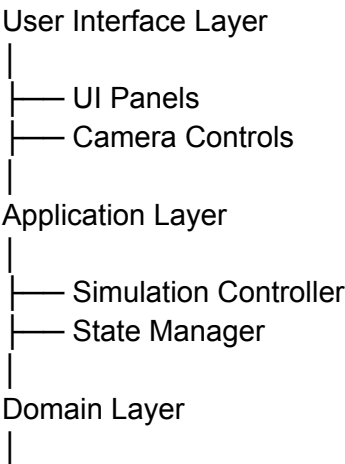
- 1. **Separation of physics and rendering**
- 2. **Single source of truth for simulation state**
- 3. **UI observes simulation state but never modifies physics directly**

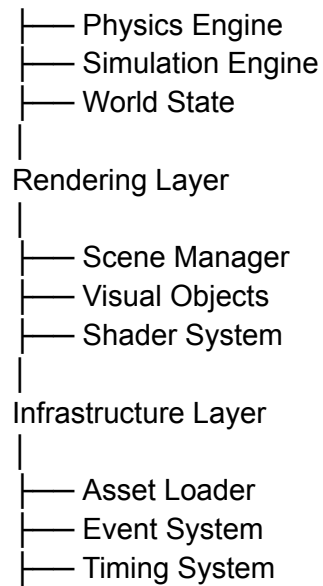
This prevents tight coupling and keeps the system maintainable.

Architecture style:

Clean Architecture + Simulation Engine Pattern

2. High-Level System Architecture





Each layer only communicates downward.

3. Core Modules

The simulator will contain the following major modules.

```
core
physics
simulation
render
ui
systems
assets
config
tests
```

Responsibilities are clearly separated.

4. Directory Structure

```
black-hole-simulator
```

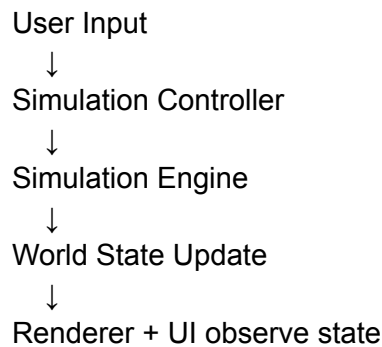
```
src
```

- core
 - constants
 - math
 - units
- physics
 - schwarzschildPhysics
 - redshift
 - timeDilation
- simulation
 - worldState
 - simulationEngine
 - objectModels
- render
 - sceneManager
 - blackHoleRenderer
 - spacecraftRenderer
 - starfieldRenderer
 - lensingShader
- ui
 - layout
 - panels
 - controls
- systems
 - cameraSystem
 - inputSystem
 - animationSystem
 - timingSystem
- assets
 - textures
 - models
 - sprites
- config
 - simulationConfig
- tests

This structure ensures the codebase remains readable.

5. Data Flow

The simulator uses **unidirectional data flow**.



Rendering never alters simulation data.

6. World State Model

The simulation uses a central state object.

Example structure:

WorldState

globalTime

blackHole

mass

schwarzschildRadius

spacecraftNear

distance

position

velocity

dilationFactor

redshiftFactor

spacecraftFar

distance

clocks

nearObserverTime

farObserverTime

All systems read from this object.

7. Simulation Engine

The simulation engine updates the world state every step.

Main function:

updateSimulation(dt)

Internal sequence:

readInput()

updateSpacecraftPosition()

computeTimeDilation()

computeRedshift()

updateClocks()

emitUpdatedState()

8. Physics Engine Responsibilities

The physics module calculates relativistic effects.

Functions:

computeSchwarzschildRadius()

computeTimeDilation()

computeRedshift()

computeSpaghettificationFactor()

All equations remain isolated in this module.

9. Rendering Architecture

Rendering uses **Three.js**.

Key rendering components:

SceneManager
CameraController
BlackHoleRenderer
SpacecraftRenderer
StarfieldRenderer
LensingShader

The renderer receives data from the world state.

10. Scene Graph

Scene objects:

Scene

BlackHole
AccretionDisk
Starfield
SpacecraftFar
SpacecraftNear
Camera

All objects are children of the main scene.

11. Camera System

Camera type:

Orbital camera.

Capabilities:

orbit
zoom
rotate
focus black hole

Limits prevent entering the event horizon.

12. Shader System

A shader will simulate gravitational lensing.

Effect:

- starfield distortion
- Einstein ring formation

The shader receives:

blackHolePosition
observerDistance
lensStrength

This produces visual warping.

13. Event System

The simulation emits events when important thresholds occur.

EVENT_SAFE_REGION
EVENT_STRONG_GRAVITY
EVENT_HORIZON_APPROACH
EVENT_HORIZON_CROSS
EVENT_SPAGHETTIFICATION

These trigger visual transitions.

14. Simulation Clock

Two time systems exist.

realTime
simulationTime

Simulation time is scaled:

$\text{simulationTime} = \text{realTime} \times \text{simulationSpeed}$

Clocks diverge based on relativistic effects.

15. Performance Strategy

Performance targets:

60 FPS
< 20k polygons
minimal shader load

Optimization techniques:

low-poly assets
GPU-based distortion
instanced starfield rendering

16. User Interface Components

The UI will be implemented using **React**.

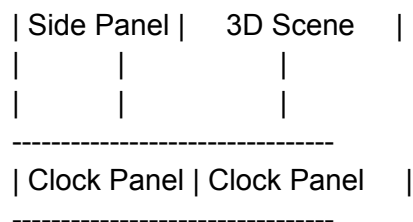
Major components:

SidePanel
EquationPanel
ControlPanel

ClockPanel
ViewSelector

These components subscribe to world state updates.

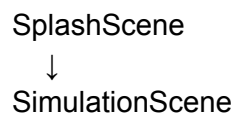
17. UI Layout



The main focus remains the simulation.

18. Splash Screen

Startup sequence:



Splash contains:

- developer logo
- project title
- fade animation

Duration: ~3 seconds.

19. Testing Strategy

Physics modules require unit testing.

Tests include:

Schwarzschild radius calculation

time dilation accuracy

redshift behavior near horizon

Testing ensures scientific correctness.

20. Development Roadmap

The project will follow a strict phase order.

Phase 1

Physics engine

Phase 2

Simulation engine

Phase 3

Clock system

Phase 4

Basic 3D rendering

Phase 5

Spacecraft interaction

Phase 6

Redshift visualization

Phase 7

Spaghettification effects

Phase 8

Gravitational lensing shader

Phase 9

User interface

Phase 10

Optimization and testing

21. Future Extensions

Possible upgrades:

Kerr rotating black holes

gravitational wave visualization

photon orbit simulation

multiple observers

These are outside the initial scope.

22. Final Project Identity

Project Name:

Interactive Schwarzschild Black Hole Simulator

Category:

Interactive physics visualization.

Purpose:

Educational exploration of relativistic phenomena near black holes.

Author:

Natalia.